The Journal of Systems and Software 231 (2026) 112634

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Do comments and expertise still matter? An experiment on programmers' adoption of AI-generated JavaScript code☆

Changwen Li [a],*, Christoph Treude [b], Ofir Turel [a],*

[a] The University of Melbourne, Grattan Street, Melbourne, 3010, Victoria, Australia
[b] Singapore Management University, 81 Victoria Street, Singapore, 188065, Singapore

ARTICLE INFO

Keywords:
AI programming assistant
Empirical software engineering
Technology adoption
Human–computer interaction

ABSTRACT

This paper investigates the factors influencing programmers' adoption of AI-generated JavaScript code recommendations within the context of lightweight, function-level programming tasks. It extends prior research by (1) utilizing objective (as opposed to the typically self-reported) measurements for programmers' adoption of AI-generated code and (2) examining whether AI-generated comments added to code recommendations and development expertise drive AI-generated code adoption. We tested these potential drivers in an online experiment with 173 programmers. Participants were asked to answer some questions to demonstrate their level of development expertise. Then, they were asked to solve a LeetCode problem without AI support. After attempting to solve the problem on their own, they received an AI-generated solution to assist them in refining their solutions. The solutions provided were manipulated to include or exclude AI-generated comments (a between-subjects factor). Programmers' adoption of AI-generated code was gauged by code similarity between AI-generated solutions and participants' submitted solutions, providing a behavioral measurement of code adoption behaviors. Our findings revealed that, within the context of function-level programming tasks, the presence of comments significantly influences programmers' adoption of AI-generated code regardless of the participants' development expertise.

1. Introduction

One of the most common hurdles in software engineering is turning ideas into programs (Mohagheghi and Conradi, 2007). A code generation system capable of automatically translating natural languages into code could assist programmers, leading to a more efficient and productive programming paradigm (Li et al., 2022; Xu et al., 2022). Automatically generating code from high-level descriptions has been a historic challenge for decades in the computer science community (Manna and Waldinger, 1971). The recent breakthrough in artificial intelligence, especially in large language models (LLMs), brings the computer science community closer to addressing this challenge (Li et al., 2022; Barke et al., 2023). AI has promising potential to revolutionize software engineering processes, including code search, code recommendation, automatic bug fixing, code review, risk assessment of code changes, and troubleshooting (Bader et al., 2021).

Despite the fact that recent advancements in artificial intelligence are reshaping software engineering (Barke et al., 2023; Feng et al., 2023), little research has investigated the factors that influence programmers' adoption of AI-generated code (Russo, 2024). Ge and Wu (2023) conducted an interview-based qualitative study to investigate the factors that impact the adoption of ChatGPT for bug fixing. They discovered several key influential factors, including performance expectancy, effort expectancy, social influence, facilitating conditions, data security, and trust (Ge and Wu, 2023). Russo conducted a survey-based study to investigate the factors influencing developers' adoption of generative AI tools in software engineering (Russo, 2024). Russo proposed the Human-AI Collaboration and Adaptation Framework (HACAF), a theoretical framework to explain programmers' adoption of AI coding tools (Russo, 2024). Liang et al. conducted a large-scale survey to evaluate the reasons programmers choose to use or avoid AI coding assistants by directly asking participants to answer why they use or refuse AI coding tools (Liang et al., 2024).

While previous studies exploring the factors influencing programmers' adoption of AI-generated code have made valuable contributions, several research gaps remain. The first gap pertains to the over-reliance on self-reported data. The assessment of participants' adoption of AI was solely based on self-reported data, lacking complementary objective measurements. Previous studies used participants' self-reported

☆ Editor: Dario Di Nucci.

* Corresponding authors.
E-mail addresses: changwenl@student.unimelb.edu.au (C. Li), ctreude@smu.edu.sg (C. Treude), oturel@unimelb.edu.au (O. Turel).

https://doi.org/10.1016/j.jss.2025.112634
Received 12 March 2025; Received in revised form 24 August 2025; Accepted 8 September 2025
Available online 23 September 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

intentions to gauge their adoption of AI-generated code or AI coding tools. For example, Russo used the question, "How likely are you to use a language model in your work in the next six months?" to measure participants' willingness to adopt AI coding tools (Russo, 2024). The reasons participants adopt or avoid AI-generated code were also collected through self-reported data by directly querying the reasons rather than conducting controlled experiments (Liang et al., 2024; Russo, 2024). Some researchers (Donaldson and Grant-Vallone, 2002; Van de Mortel, 2008; Chan, 2010) have raised concerns about the limitations of self-reported questions in surveys. Self-reported data may not fully align with the actual usage pattern since self-reported data are subjective perceptions rather than objective facts.

The second research gap is the non-comprehensive scope of the evaluated factors. Previous studies have primarily examined how productivity and programmers' perceptions influence the adoption of AI-generated code, neglecting some potentially crucial characteristics of programmers and the generated code. For instance, factors such as the presence of comments in AI-generated code and programmers' development expertise have been ignored.

To address the first research gap, we have designed a survey-based experiment employing objective measurements to assess participants' adoption of AI coding suggestions. In the survey, participants were required to complete a JavaScript programming task with the assistance of a code snippet generated by AI. This practical approach allows us to objectively assess their adoption of AI-generated code based on their modifications and usage of the provided snippet. Instead of merely asking participants to express their intentions to adopt AI-generated code, we analyze the submitted code to objectively determine adoption in controlled scenarios. We then conduct statistical analyses, treating potential influential factors as independent variables and the degree of adoption as the dependent variable.

To bridge the second research gap, we undertook a systematic classification of the potential influential factors and evaluated the influence of some unexplored factors. We categorize the potential influential factors into two groups: (A) *program factors* and (B) *programmer factors*. Program factors refer to the factors inherent to the program itself, representing the characteristics of the AI-generated code. Programmer factors are tied to the attributes and conditions of the individual programmer. To keep the number of factors manageable in this study, we chose to investigate the presence of comments in the AI-generated code as the program factor and the programmers' development expertise as the programmer factor.

Our methodology not only deepens our understanding of the determinants of AI code adoption but also provides objective measurements of code adoption. Objective data is not necessarily superior to subjective self-reported data, but it could provide valuable complements to prior research.

2. Related work

In this section, we introduce existing studies investigating factors that influence programmers' adoption of AI-generated code and AI coding tools. We also introduce the independent variables in this research: the presence of comments and development expertise, along with the control variable of attitude toward AI code generators. Since the influence of comments and development expertise on programmers' adoption of AI-generated code has not been studied before, we focus on introducing related work on their influence on code comprehension.

2.1. Factors influencing programmers' adoption

There is very little research examining the factors that influence programmers' adoption of AI code generators or AI-generated code. Previous related studies used interviews and surveys to investigate what impacts participants' adoption. Ge and Wu conducted semi-structured interviews with 50 participants to scrutinize factors influencing programmers' adoption of ChatGPT for bug fixing (Ge and Wu, 2023). The Unified Theory of Acceptance and Use of Technology (UTAUT) (Venkatesh et al., 2003), a model used to explain users' intentions to adopt a technology, was adopted by Ge and Wu as their theoretical foundation (Ge and Wu, 2023). Their interviews included both open-ended and focused questions about how the four determinants in UTAUT, including performance expectancy, effort expectancy, social influence, and facilitating conditions, affect participants' adoption of AI-generated bug-fixing suggestions (Ge and Wu, 2023). Their study indicated that all of the four determinants in UTAUT, together with data security and programmers' trust, could influence the adoption of ChatGPT for bug fixing (Ge and Wu, 2023).

Russo conducted a survey-based study with 183 participants, using nine open-ended questions to investigate the factors influencing developers' adoption of generative AI tools in software engineering (Russo, 2024). Russo proposed the Human-AI Collaboration and Adaptation Framework (HACAF) to explain the adoption of generative AI tools in software engineering. HACAF is a theoretical model that combines insights from the Technology Acceptance Model (TAM) (Marangunić and Granić, 2015), Diffusion of Innovation (DOI) (García-Avilés, 2020), and Social Cognitive Theory (SCT) (Bandura, 1989) to explain programmers' adoption of generative AI tools. HACAF emphasizes four determinant factors to programmers' adoption of AI coding tools, including perceptions about the technology, compatibility factors, social factors, and personal and environmental factors (Russo, 2024). Perceptions about the technology include perceived utility and ease of use. Compatibility factors measure the extent to which AI coding tools are compatible with current software engineering practices. Social factors include social influence and self-efficacy. Personal and environmental factors are personal willingness to experiment with AI coding tools and organizational support.

Liang et al. also explored why programmers use or do not use AI code generators by conducting a survey-based study with 410 participants (Liang et al., 2024). Different from Russo's study (Russo, 2024), Liang et al. employed Likert-scale questions to assess the importance of various determinants that either motivate or discourage participants from using AI coding assistants (Liang et al., 2024). Their study indicates that the most significant reasons developers use AI code generators are the capability of AI tools to enhance productivity, including reducing keystrokes, accelerating programming, and assisting with syntax recalling. The pivotal reasons developers refuse to use AI coding tools are the inability of AI-generated code to fulfill specific requirements and the lack of control over producing desired outputs (Liang et al., 2024).

2.2. Comments

Although the impact of comments on the adoption of AI-generated code has not been systematically studied, related work on the impact of comments on code comprehension inspired our work. The significance of comments is emphasized in many coding guidelines because comments have a profound impact on code comprehension (Martin, 2009). Several empirical studies have explored the influence of comments on code comprehension (Takang et al., 1996; Nurvitadhi et al., 2003; Börstler and Paech, 2016). An early empirical study conducted by Takang et al. investigated whether the presence of comments influences code comprehension. Its findings demonstrated that comments play a profound role in enhancing program comprehension, regardless of the identifier naming convention (Takang et al., 1996). Nurvitadhi et al. expanded the study by examining how different types of comments influenced code comprehension (Nurvitadhi et al., 2003). Nurvitadhi et al. investigated how class comments and method comments influence Java code understanding. Method comments notably enhance comprehension, while class comments do not have a significant impact (Nurvitadhi et al., 2003). Börstler and Paech further expanded the research by exploring how the quality of comments influences code comprehension (Börstler and Paech, 2016). Their finding suggested that the mere presence of comments could enhance programmers' perception of the readability of the code regardless of the quality of the comments (Börstler and Paech, 2016).

2.3. Development expertise

While previous studies have not thoroughly evaluated how development expertise influences programmers' adoption of AI-generated code, related research on its impact on code comprehension offers a valuable starting point. Development expertise comprises multiple aspects, including general programming experience, programming experience in particular languages, familiarity with task-specific knowledge, and personality traits (Baltes and Diehl, 2018). Previous studies have explored the influence of development expertise on code comprehension (Lawrie et al., 2006; Wagner and Wyrich, 2022). Lawrie et al. discovered that increased years of work experience and schooling resulted in a better capability for comprehending source code (Lawrie et al., 2006). Wagner and Wyrich studied the influence of intelligence and personality traits on code comprehension (Wagner and Wyrich, 2022). The results of their study showed that there was a strong positive correlation between participants' general intelligence and their performance on code comprehension and a weaker negative correlation between participants' conscientiousness and code comprehension (Wagner and Wyrich, 2022).

2.4. Attitudes towards AI coding tools

Previous studies show that programmers have multifaceted attitudes toward AI code generators, including positive attitudes (Czarnitzki et al., 2023; Amoozadeh et al., 2024) and negative attitudes (Feng et al., 2023; Czarnitzki et al., 2023). Several studies demonstrate that some programmers trust AI code generators and believe that AI code generators could improve their productivity (Czarnitzki et al., 2023; Amoozadeh et al., 2024). Research has demonstrated that programmers adopt AI coding tools because they trust that AI coding tools could increase productivity (Liang et al., 2024; Russo, 2024). On the other hand, besides positive attitudes, programmers hold negative attitudes as well (Feng et al., 2023; Czarnitzki et al., 2023). Feng et al. examined programmers' emotional responses to ChatGPT's code-generation capabilities by gathering social media posts regarding ChatGPT's code-generation features from Twitter and Reddit (Feng et al., 2023). Their analysis showed that the most common sentiments towards ChatGPT were fear, while happiness and anger were the two least common ones (Feng et al., 2023). Feng et al. concluded that programmers' fear derives from two sources, which are the opacity of ChatGPT and the sense of job threat (Feng et al., 2023).

Kuhail et al. investigated programmers' fear of job loss (Czarnitzki et al., 2023). They investigated to what extent programmers fear job loss caused by AI code generators and why they fear or do not fear potential job loss. Participants in their study were required to rate their sense of current and future job security threats. The results suggest that only 24.2% of participants consider AI code generators as non-trivial threats, categorizing them as high threats, moderate threats, or some threats. However, the outlook changes when considering the future, with approximately 50% of the participants perceiving AI code generators as non-trivial threats to their job security. Kuhail et al. also discovered a positive correlation between programmers' trust in AI code generators and their fear of job replacements (Czarnitzki et al., 2023).

While prior research has investigated various factors influencing programmers' adoption of AI-generated code and AI coding tools, none have utilized objective measurements for participants' adoption of AI-generated code. In this work, we focus on how two previously unexplored factors — AI-generated comments and developer expertise — affect the adoption of AI-generated code. By conducting an experiment, we utilized objective measurements to gauge participants' adoption of AI-generated code and examine how these factors impact programmers' adoption behaviors, thereby addressing research gaps.

3. Research methodology

3.1. Research questions

In this study, we aim to evaluate the influence of two unexplored variables on programmers' adoption of AI-generated code: the presence of comments in the code (i.e., a program factor) and the programmers' development expertise (i.e., a programmer factor). We formulate the research questions $\mathbf{RQ_1}$ and $\mathbf{RQ_2}$ accordingly.

$\mathbf{RQ_1}$. How does (A_1) *presence of comments* influence programmers' adoption of AI-generated code?

Our hypothesis was developed based on the Unified Theory of Acceptance and Use of Technology (UTAUT) (Venkatesh et al., 2003). Fig. 1 illustrates how the presence of comments shapes programmers' adoption intentions and how developers' expertise moderates the effect of comments on adoption intentions, with all relationships grounded in UTAUT. According to UTAUT, lower effort expectancy is linked to higher adoption intention (Venkatesh et al., 2003). Moreover, existing research has underscored the critical role of comments in supporting code comprehension (Martin, 2009). Building on these theoretical and empirical foundations, we propose that the inclusion of comments in AI-generated code will reduce the time cost and cognitive burdens programmers incur to comprehend the code. This reduction aligns with UTAUT's emphasis on effort expectancy, as lowered comprehension costs directly translate to decreased effort expectancy, ultimately influencing adoption intentions. Our formal hypothesis for $\mathbf{RQ_1}$ is:

$\mathbf{H_1}$: The factor (A_1) *presence of comments* increases programmers' adoption of AI-generated code.

$\mathbf{RQ_2}$. How does (B_1) *development expertise* moderate the influence of (A_1) *presence of comments* on programmers' adoption of AI-generated code?

Programmers with richer development expertise tend to be more experienced in programming and more familiar with task-specific knowledge (Baltes and Diehl, 2018), which could potentially enable them to have a better understanding of AI-generated code without comments. Aligning with UTAUT, experienced programmers — equipped with stronger code comprehension abilities — perceive a smaller reduction in effort expectancy when comments are present. This is because the reductions in time costs and cognitive burdens facilitated by comments hold less significance for them, given their capacity to effectively comprehend code regardless of whether comments are included. We hypothesize that the influence of (A_1) *presence of comments* on programmers' adoption of AI-generated code would be moderated by (B_1) *development expertise*, such that the positive effect of comments on adoption decreases as development expertise increases. The formal hypothesis for $\mathbf{RQ_2}$ is:

$\mathbf{H_2}$: The positive effect of (A_1) *presence of comments* on programmers' adoption of AI-generated code decreases as a function of (B_1) *development expertise*.

3.2. Experimental design

3.2.1. Experiment structure

To ensure the reliability of our findings, we designed a survey-based experiment with a structured approach. By having participants first solve a programming problem on their own, we create a baseline for their understanding and approach. Then, by introducing an AI-generated solution with or without comments, we can objectively measure how these factors influence their adoption behaviors.

This methodology allows us to isolate the effects of comments and development expertise, providing a clear understanding of how these specific factors impact programmers' willingness to adopt AI-generated code. Additionally, this design discourages participants from passively accepting AI-generated solutions, prompting them to critically

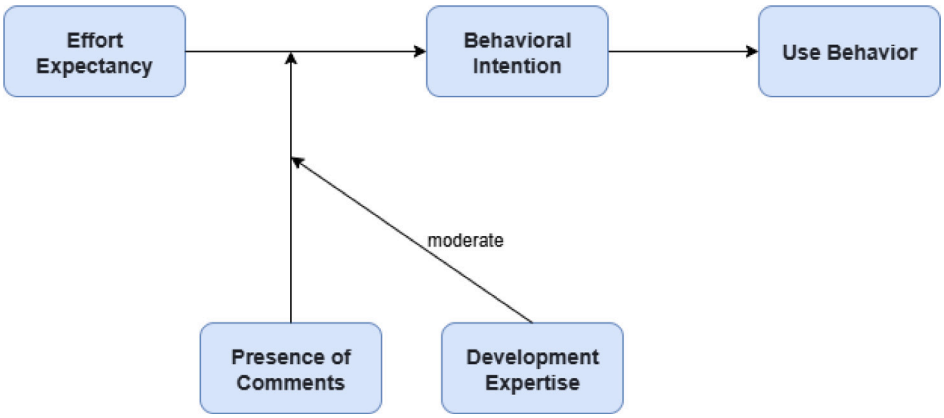

Fig. 1. Hypothesis model based on UTAUT.

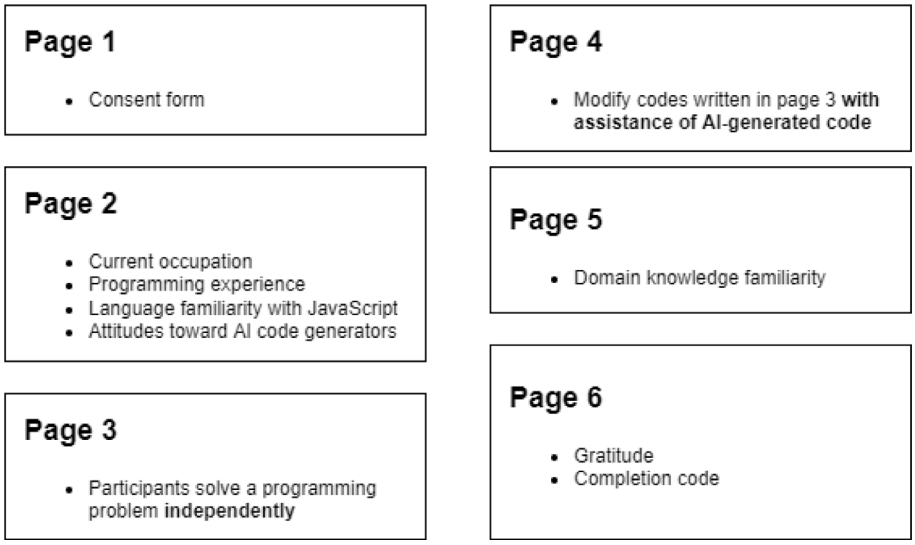

Fig. 2. The outline of the survey structure.

evaluate and potentially integrate them with their own coding efforts. Conducting the experiment online with a consistent environment helps maintain control over external variables, ensuring that our results are robust and generalizable.

We developed a custom web application to host the survey on https://survey.changwen-software-engineering.xyz. We conducted pilot tests with five professional developers to ensure that there were no technical errors or usability issues in the survey. Participants in the pilot tests were recruited through the authors' professional network to ensure that they treated the study seriously and provided quality feedback. We conducted each run with a different participant and fixed the problems encountered in each run.

The survey included six pages in total. The outline of the survey structure is presented in Fig. 2. The first page requested the participants' consent to participate in the study. Page one provided statements of consent forms, researchers' contact information, and a link to the plain language statement (PLS). After participants had opted to participate in the survey, the survey would navigate to the second page.

The second page collected participants' demographics to capture the raw data for programmer factors, including (B_{11}) *programming experience*, (B_{12}) *programming language proficiency*, and (B_4) *attitudes toward AI code generators*. We postponed measuring (B_{13}) *domain knowledge familiarity* until page five to avoid biasing participants. Domain knowledge is the knowledge utilized to solve the programming problem on the third and fourth pages. Asking about domain knowledge upfront could unintentionally reveal what concepts were relevant to the upcoming programming task, influencing participants' responses. For example, if the task involved tree structures, asking about familiarity with trees in advance could hint at the solution and influence participants' responses.

On the third page, we showed participants the programming problems and required them to complete the programming problem in JavaScript **on their own**. We required participants to solve the programming problem independently first for two reasons. Firstly, it motivated participants to actively engage by trying to solve the problem themselves. In pilot tests, we observed that without this step, many participants would copy AI-generated solutions without fully comprehending the questions. Secondly, it helped us exclude potentially invalid responses, as participants unwilling to put in effort tended to quit the survey at this stage. This self-exclusion helped filter out less meaningful responses and improve data quality. We limited the programming language to JavaScript due to the limited participant number, ensuring manageable feasibility in the study.

The screenshot of the third page is shown in Fig. 3. As illustrated, the coding problem interface consists of several distinct regions, including the code editor region, test case region, timer region, and problem description region.

To provide participants with a coding experience similar to Microsoft Visual Studio Code (VSCode) (Del Sole and Sole, 2019), we integrated Monaco Editor, the code editor behind VSCode (Microsoft,

Fig. 3. Overview of page three.

2024), into the code editor region. This integration enabled features such as syntax highlighting and auto-completion, allowing participants to quickly adapt to the environment.

In the test case region, participants could execute their code using either custom test cases or predefined ones. Each programming problem included ten predefined test cases, which were hidden from participants. As shown in Fig. 4, the test case region also supported switching to an output console, where participants could view comprehensive execution results, including error types, error messages, and the line number where an error occurred.

A time limit was imposed on this page, ranging from a minimum of one minute to a maximum of ten minutes. The lower bound encouraged participants to make a genuine effort, while the upper bound simulated real-life time constraints and helped maintain a manageable study schedule.

On the fourth page, we provided a code snippet and notified the participants that it was generated by an AI code generator. They were also notified that they could modify the code written by themselves on page three with the assistance of the provided AI-generated code. Participants were explicitly informed that they had the option to use all the provided code, integrate part of the provided code into their original code, or choose not to use the provided code at all. We employed the same timer, code editor, and code execution features on page three.

The fifth page of the survey gathered information on participants' familiarity with the domain knowledge (B_{13}).

On the sixth page, we expressed our gratitude to the participants. We provided a completion code for them and guided them to provide it so that we knew that they had finished the study. After receiving the completion code, we would pay them.

3.2.2. Motivation and engagement mechanisms

To motivate participants to focus on doing this survey and put in more effort, we have utilized two methods, including bonus payment and full-screen mode. For the bonus payment mechanism, we informed the participants that their reimbursement depended on the quality of the code they submitted on page three and page four. We told participants that they would only receive 2.6 GBP if the quality of their code was low and that they would receive 1 additional GBP as a bonus award if they submitted high-quality code. The 1 GBP payment increase was significant in this study because it was a 38% increase in basic payment. In the end, we gave every participant both the basic 2.6 GBP reimbursement and the 1 GBP bonus regardless of the quality of their submitted code because it is hard to objectively determine the quality of their submitted code.

There were two purposes for adopting the bonus payment mechanism. The first purpose was to ensure genuine engagement from participants. Performance-based incentives could help increase participants' performance (Brase, 2009). The difference in payment could motivate participants to put more effort into their participation. The second purpose was to motivate participants to provide more representative data. Programmers are often required to write high-quality code in their daily work. Therefore, motivating participants to produce high-quality code in our survey was a way to motivate them to approach the task as if they were coding in their actual professional work.

The second motivation mechanism was full-screen mode, which encouraged participants to stay in full-screen mode while doing the survey. We motivated participants to stay in full-screen mode for two reasons. Firstly, staying in full-screen mode helps participants focus on the survey. Secondly, maintaining the survey in full-screen mode helps to prevent participants from seeking external assistance, such as

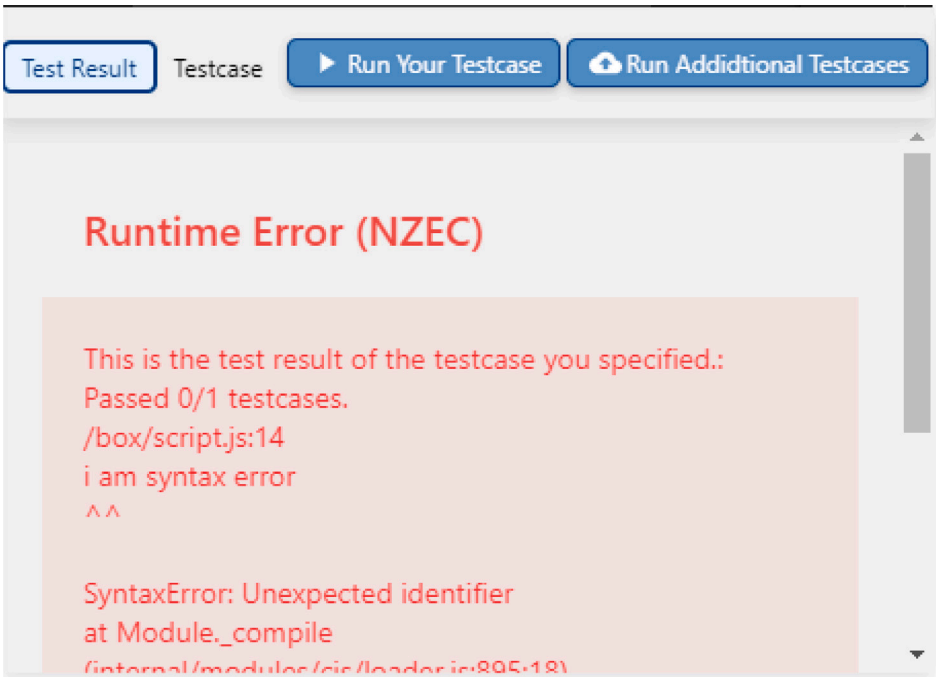

Fig. 4. Overview of the test case region.

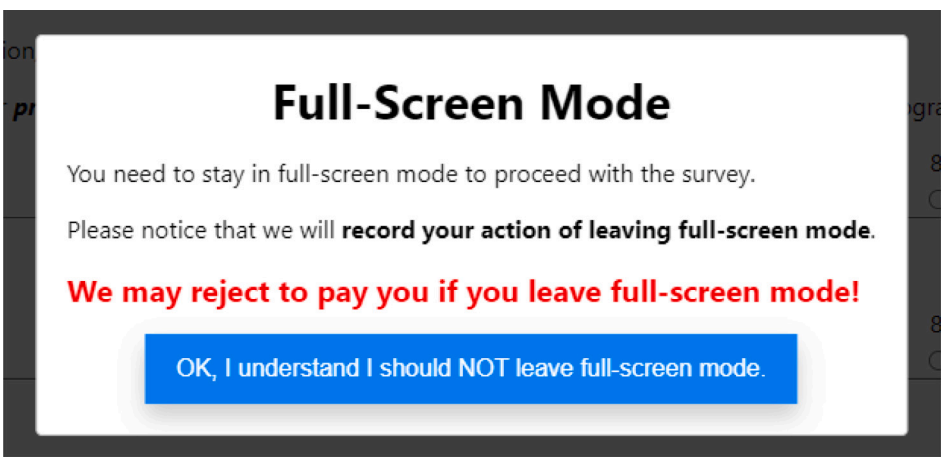

Fig. 5. Full-screen mode dialogue window while not in full-screen mode.

information from the Internet and solutions created by the participants themselves using AI tools. Our goal was that participants only interact with the AI-generated solutions provided in our survey to gauge their adoption accurately. External information could influence their adoption of the provided AI-generated code in the survey, which might introduce bias.

We used a dialogue window to guide participants to stay in full-screen mode. After participants signed the consent form, they would see a dialogue window as shown in Fig. 5. This dialogue window informed the participants that they needed to stay in full-screen mode, and their behavior of leaving full-screen mode would be recorded. We also warned them that leaving full-screen mode may lead to no reimbursement, although we still paid them even if they had left full-screen mode. We recorded this data to enable us to discard participant data that may have been tainted by seeking outside assistance. Once participants clicked the OK button, their browsers were switched to full-screen mode. Once they left full-screen mode, this dialogue window shown in Fig. 5 would appear again. This survey design was approved by the ethics review board of our institution and follows the rules of the Prolific data collection platform used in this work.

3.3. Experimental objects

The experimental objects contained programming problems, the corresponding predefined test cases, and AI-generated solutions to the programming problems.

3.3.1. Programming problems

We have chosen five programming problems from LeetCode, a well-known and popular platform where programmers practice programming (Nguyen and Nadi, 2022a; Harper, 2023). Each survey will be randomly assigned one of these five programming problems and the participant needs to solve this programming problem on the third page and the fourth page of the survey. We chose programming problems from LeetCode for the following two reasons. Firstly, LeetCode's popularity increases the chances of high-quality and correct programming problems. The programming problems from LeetCode have clear introductions and well-crafted content. Moreover, LeetCode programming problems have been proven to be good experimental materials, as

Table 1
A brief summary of selected programming problems.

Name	Topics	Difficulty level	LeetCode number[a]
Ransom Note	Map	Easy	383
Remove Duplicates from Sorted List	Linked List	Easy	83
Find the Winner of the Circular Game	Queue	Medium	1823
Validate Binary Search Tree	Tree	Medium	98
Longest Valid Parentheses	Stack	Hard	32

[a] LeetCode Number is a unique identifier assigned to each problem on the LeetCode platform for easy reference and search.

they have been used in multiple studies to evaluate AI code generation (Czarnitzki et al., 2023; Nascimento et al., 2023; Nguyen and Nadi, 2022b).

To enhance the generalizability of this study, we chose five programming problems with different topics and different difficulty levels. According to the topic tags provided by LeetCode, the topics of these programming problems include map, stack, queue, linked list, and tree. We have chosen two easy programming problems, two medium programming problems, and one hard programming problem based on the difficulty labeled by LeetCode. A brief summary of the five selected programming problems is presented in Table 1.

3.3.2. Predefined test cases

Participants were able to execute their code using predefined test cases on both page three and page four of the survey. Each programming problem included 10 pre-defined test cases, all of which were hidden from participants. These test cases were generated by GPT 4 and subsequently reviewed by the authors of this paper to ensure their correctness and scalability.

3.3.3. AI-generated solutions

We have created two versions of AI-generated solutions for each programming problem: solutions **with** comments and solutions **without** comments. Every survey was assigned one of these two versions. On the fourth page of the survey, participants used the assigned AI-generated solutions to assist in modifying the code written by themselves on page three.

The provided AI-generated code snippets were generated by ChatGPT 4 Turbo. To generate the solutions, we copied the problem description, including examples, into ChatGPT 4 Turbo. Then, we added the prompt "Please solve this problem in JavaScript" and included the template code from LeetCode, which provides function signatures. We then added a final prompt based on the version: "Please do not add any comment" for the no-comment version and "Please add meaningful comments" for the commented version. We confirmed that all the code snippets generated by ChatGPT in this research could be executed correctly and passed all test cases on LeetCode. We also verified that all comments in the AI-generated code were meaningful and contributed to understanding the solution, rather than being irrelevant or nonsensical. An example of the AI-generated solution with and without comments is shown in Fig. 6.

3.4. Variables, measurements, and analyses

Table 2 demonstrates the variables in this study, including independent variables, control variables, and dependent variables.

3.4.1. Independent variables

The independent variables in this research were (A_1) *the presence of comments* and (B_1) *development expertise*. The variable A_1 was a categorical variable determined by whether the AI-generated code included comments. The value of A_1 was 1 if comments were present, and the value of A_1 was 0 if comments were absent.

To capture the complexity of (B_1) *development expertise*, the corresponding factor B_1 has three sub-factors, including (B_{11}) *programming experience*, (B_{12}) *programming language proficiency*, and (B_{13}) *domain knowledge familiarity*. We assessed (B_1) *development expertise* by performing PCA (Principal Component Analysis) on B_{11}, B_{12}, and B_{13}. The detailed PCA analysis will be introduced in Section 4.1.1.

To enhance the reliability of the obtained data, we utilized two questions to assess each sub-factor. First, we employed a Likert scale question (B_{11a}, B_{12a}, B_{13a}) for each sub-factor, translating participants' perceptions into integers from 1 (Not experienced/proficient/familiar at all) to 5 (Extremely experienced/proficient/familiar). For sub-factors B_{11} and B_{12}, we used questions B_{11b} and B_{12b} to gauge participants' years of experience in overall programming and JavaScript development, respectively. The second question used to assess B_{13}, question B_{13b}, measured participants' familiarity with specific domain knowledge by determining their level of prior exposure to certain topics. The possible levels were "I have never heard of x", "I have heard of x but never used it", "I have occasionally used x", and "I have regularly used x" (here, x represents a certain domain knowledge topic).

3.4.2. Control variables

The control variables are (B_2) *gender*, (B_3) *age*, and (B_4) *attitudes towards AI Code Generators*. We evaluated (B_4) *attitudes towards AI Code Generators* by mirroring and modifying the Attitudes Towards Artificial Intelligence (ATAI) scale proposed by Sindermann et al. (2021). Akin to the ATAI scale, we categorized participants' attitudes toward AI code generators into two domains: fear and attitudinal acceptance. Therefore, we had two sub-factors for B_4, which are (B_{4f}) *fear of AI code generators* and (B_{4a}) *attitudinal acceptance of AI code generators*. We explored B_4 through grid Likert scales in Fig. 7, where the mean value of the responses to the first two questions represents B_{4a}, and the mean value of the responses to the last three questions represents B_{4f}.

3.4.3. Dependent variable

Building on research on algorithm appreciation (Logg et al., 2019), which suggests that individuals often rely on algorithmic outputs even without consciously acknowledging their influence, we define code adoption as **behavioral reliance on AI-generated code** rather than verbal declaration. Consistent with prior research that favors observable behavior (Logg et al., 2019), we measure this reliance using code similarity metrics—that is, the extent to which participants incorporated AI-generated code into their final submissions.

To operationalize this definition, we devised a novel objective metric. Participants were tasked with writing a code snippet with the assistance of an AI-generated solution (page four of the survey). The degree of adoption was then quantified by comparing the similarity between the code submitted by the participants and the AI-generated code provided on page four. This method provides an objective assessment of the code adoption behavior.

To gain insights into the dependent variable from different dimensions, four distinct code similarity metrics were utilized in this study, including line-level code similarity from the participant perspective (*LP*), line-level code similarity from the AI perspective (*LA*), token-sequence-level code similarity from the participant perspective (*TP*), and token-sequence-level code similarity from the AI perspective (*TA*). These metrics could be categorized by two dimensions: the dimension of entity perspective and the dimension of code similarity level. Table 3 shows the classification and interpretation of these metrics. For data analysis, we created a new latent variable, *latent code similarity*, by combining *LP*, *LA*, *TP*, and *TA* through PCA. The detailed PCA analysis

```javascript
/**
 * @param {string} s
 * @return {number}
 */
var longestValidParentheses = function (s)
     {
  let maxLen = 0;
  let stack = [-1];

  for (let i = 0; i < s.length; i++) {
    if (s[i] === "(") {
      stack.push(i);
    } else {
      stack.pop();
      if (stack.length === 0) {
        stack.push(i);
      } else {
        const currentLen = i - stack[stack
    .length - 1];
        maxLen = Math.max(maxLen,
    currentLen);
      }
    }
  }

  return maxLen;
};
```

Listing 1: An example of the AI-generated code without comments

```javascript
/**
 * @param {string} s
 * @return {number}
 */
var longestValidParentheses = function (s)
     {
  let maxLen = 0;
  // Use a stack to track the indices of
    unmatched parentheses.
  // -1 handles edge cases where the
    substring starts at index 0
  let stack = [-1];

  for (let i = 0; i < s.length; i++) {
    if (s[i] === "(") {
      stack.push(i);
    } else {
      // Pop for matching ')'
      stack.pop();
      if (stack.length === 0) {
        // If stack is empty, it means
    this ')' is unmatched.
        // Push its index to reset the
    base for future valid substrings.
        stack.push(i);
      } else {
        // Calculate current valid
    substring length
        const currentLen = i - stack[stack
    .length - 1];
        maxLen = Math.max(maxLen,
    currentLen);
      }
    }
  }

  return maxLen;
};
```

Listing 2: An example of the AI-generated code with comments

Fig. 6. An example of the AI-generated code with and without comments.

To what extent do you agree with the following statements?

	0 (Strongly disagree)	1	2	3	4	5	6	7	8	9	10 (Strongly agree)
I trust AI code generation system	○	○	○	○	○	○	○	○	○	○	○
AI code generation system will benefit programmers	○	○	○	○	○	○	○	○	○	○	○
I fear AI code generation system	○	○	○	○	○	○	○	○	○	○	○
AI will destroy humankind	○	○	○	○	○	○	○	○	○	○	○
AI code generation system will cause many job losses	○	○	○	○	○	○	○	○	○	○	○

Fig. 7. Grid Likert-scale question on evaluating participants' attitudes towards AI code generators (Sindermann et al., 2021).

is presented in Section 4.1.2. The new latent variable, *latent code similarity*, was the dependent variable for hypotheses testing.

The dimension of entity perspective indicated the entity from which the similarity metric was assessed, suggesting which entity was used as the denominator when calculating similarity. The possible values for the entity perspective dimension were participant and AI. If the entity perspective of a code similarity metric is "participant", then the metric represents the percentage of the participant's code that is similar to the AI-generated code, highlighting to what extent the participant's coding approach has been influenced by the AI-generated solution.

Table 2
Outline of variables.

Abbreviation	Full name	Category
A_1	Presence of comments[a]	Independent variable
B_1	Development expertise[b]	Independent variable
B_{11a}	Perceptions of programming experience	Independent variable
B_{11b}	Years of overall programming experience	Independent variable
B_{12a}	Perceptions of programming language proficiency	Independent variable
B_{12b}	Years of experience in JavaScript development	Independent variable
B_{13a}	Perceptions of domain knowledge familiarity	Independent variable
B_{13b}	Prior exposure to domain knowledge	Independent variable
B_2	Gender[c]	Control variable
B_3	Age[d]	Control variable
B_4	Attitudes toward AI code generators[e]	Control variable
B_{4f}	Fear of AI code generators	Control variable
B_{4a}	Attitudinal acceptance of AI code generators	Control variable
Y	Latent code similarity[f]	Dependent variable

[a] The variable "presence of comments" is manipulated by providing AI-generated code snippets either with comments or without them.
[b] The variable "development expertise" is obtained through PCA analysis on B_{11a}, B_{11b}, B_{12a}, B_{12b}, B_{13a}, and B_{13b}.
[c] Data of the variable "gender" is provided by Prolific.
[d] Data of the variable "age" is provided by Prolific.
[e] Attitudes toward AI code generators comprise two aspects, which are fear and attitudinal acceptance.
[f] Latent code similarity is computed through PCA analysis on four different code similarity metrics in Table 3.

Table 3
Classification of code similarity metrics.

Abbreviation	Entity perspective dimension	Code similarity level dimension	Interpretation
LP	Participant	Line level	Percentage of the participant's code that are similar to the AI-generated code at the line level.
LA	AI	Line level	Percentage of the AI-generated code that is similar to the participant's code at the line level.
TP	Participant	Token sequence level	Percentage of the participant's code that are similar to the AI-generated code at the token sequence level.
TA	AI	Token sequence level	Percentage of the AI-generated code that are similar to the participant's code at the token sequence level.

If the entity perspective of a code similarity metric is "AI", then the metric represents the percentage of the AI-generated code similar to the participant's code, revealing the percentage of AI-generated code used by the programmer.

The dimension of code similarity level showed the level at which the comparison was conducted. The possible values for the dimension of code similarity level were line level and token sequence level. The line-level code similarity metrics calculated the ratio of identical lines to total lines. Only lines that were exactly the same in both the AI-generated code and the participants' submitted code would be counted as a match (differences in whitespace and comments were ignored). Token-sequence-level code similarity metrics compute similarity by comparing the token sequences of two code snippets, a process carried out using Jplag (Prechelt et al., 2002). The token is a basic element of the code abstraction (Prechelt et al., 2002; Schleimer et al., 2003). A token sequence represents some fractions of the code snippet without necessarily representing a line of code. Token sequence abstracts the code, providing a mapping to the code structure without being influenced by different identifier names and comments (Schleimer et al., 2003).

Some code similarity-checking tools were developed to detect plagiarism in students' programming assignments at a token-sequence level (Lancaster and Culwin, 2004), including MOSS (Schleimer et al., 2003) and JPlag (Prechelt et al., 2002). We used JPlag (Prechelt et al., 2002) to calculate the code similarity at the token-sequence level[1]. Line-level code similarity investigated participants' direct copy-and-paste behavior. In contrast, the token-sequence-level code similarity focused on the abstract structure of the code.

JPlag captures code similarity at an abstract, structural level by detecting patterns such as iteration loops, control flow, and the use of data structures in the AI-generated code (Prechelt et al., 2002). This enables JPlag to identify participants' adoption of AI-generated algorithms even when the code is not copied verbatim but reimplemented with similar underlying logic. For example, in the case shown in Fig. 8, both the AI-generated code and the participant's submission used recursion to validate a binary tree. Despite this shared algorithmic structure, the two code snippets appeared quite different on the surface, with only two exact line matches. The line-level similarity from the AI perspective (LA) was just 0.13, whereas the token-sequence-level similarity from

[1] We utilized the JPlag Java package from https://github.com/jplag/JPlag to calculate the token-sequence-level similarity

```javascript
var isValidBST = function (root) {
  function validate(node, min, max) {
    // An empty tree is a valid BST
    if (!node) return true;

    // The node's value must be within the
     min and max bounds
    if (
      (min !== null && node.val <= min) ||
      (max !== null && node.val >= max)
    ) {
      return false;
    }

    // Recursively check the left subtree
    and right subtree
    // For the left subtree, update the
    max bound
    // For the right subtree, update the
    min bound
    return (
      validate(node.left, min, node.val)
    && validate(node.right, node.val, max)
    );
  }

  // Initially, there are no bounds on the
     root's value
  return validate(root, null, null);
};
```

Listing 3: An example of the AI-generated code

```javascript
var isValidBST = function (root) {
  return isValidNode(root, -Infinity,
    Infinity);
};

function isValidNode(node, min, max) {
  if (node === null) return true;

  const curValidate = node.val > min &&
    node.val < max;

  // recursively check left subtree
  const leftValidate = isValidNode(node.
    left, min, node.val);

  // recursively check right subtree
  const rightValidate = isValidNode(node.
    right, node.val, max);

  return curValidate && leftValidate &&
    rightValidate;
}
```

Listing 4: An example of the user-submitted code

Fig. 8. A comparison between AI-generated code and user-submitted code.

the AI perspective (TA), as computed by JPlag, was 0.75. This example illustrates JPlag's ability to capture structural similarity beyond superficial textual resemblance.

3.5. Participants

Participants in this study were recruited from Prolific (Palan and Schitter, 2018), an online crowd-sourcing academic data collection platform used extensively in multiple research fields (Palan and Schitter, 2018), including software engineering (Russo, 2024; Russo et al., 2021; Russo and Stol, 2020). This study strictly followed the applicable guidelines for ethical conduct in human research and was approved by the ethics review board of our institution.

3.5.1. Pre-screening mechanism

Participants were pre-screened to ensure they met the study's eligibility criteria. We utilized pre-screeners provided by Prolific to select participants with programming experience in JavaScript to ensure they were capable of doing the programming tasks in the survey. To minimize misunderstanding of the English instructions in our survey, we further pre-screened participants to ensure their primary language was English. The last pre-screener excluded participants with approval rates below 95%. The approval rate indicates the percentage of submissions approved by researchers. A low approval rate could indicate a participant's history of negative behaviors, such as submitting low-effort responses (Schild et al., 2021). To mitigate low-effort responses in our survey, we required participants to have at least a 95% approval rate.

3.5.2. Post-survey screening mechanism

We employed two post-survey screening approaches to filter out invalid responses. The first approach was excluding responses that might have sought external assistance. If participants left full-screen mode more than twice or for longer than two seconds, then they were deemed as having sought external assistance. As illustrated in Section 3.2.2, using external resources could introduce bias to participants' adoption of the provided AI-generated code. Therefore, these responses were viewed as invalid responses and excluded from the data analysis.

The second post-survey screening approach was to exclude low-effort responses. A low-effort response has a damaging impact on survey-based research as it introduces untrustworthy data (DeSimone et al., 2018). To examine the level of effort of a response, we manually reviewed the code participants submitted on page 4. We examined whether the complexity of the submitted code was significantly low. For instance, when required to validate whether a tree is a valid binary search tree, a participant submitted a solution that directly returned true. This participant returned a hard-coded value rather than trying to solve the programming problem. An overly simplistic code that did not address the programming problem suggested low effort.

In this study, we collected responses from 251 participants in total. After employing the post-survey screening mechanism, we obtained 173 valid responses. The demographics of these valid respondents are showcased in Table 4, which highlights the diversity of the participant sample.

4. Results

4.1. Preliminary analysis

Before testing the hypotheses, we first calculated the latent variables (B_1) *development expertise* and (Y) *latent code similarity*.

Table 4
Demographic summary of survey participants.

Characteristic	Description
Gender	Male (N = 119), Female (N = 54), Others (N = 0)
Age	Minimum: 18, Maximum: 68, Mean: 27, SD: 7.748; Age distribution: <23 (25%), 23–32 (50%), > 32 (25%)
Country of Residence	South Africa (N = 52), United Kingdom (N = 18), Canada (N = 16), Portugal (N = 16), Poland (N = 10)
Country of Birth	South Africa (N = 46), United Kingdom (N = 14), Portugal (N = 14), Poland (N = 10), Canada (N = 8)
Employment Status	Employed full-time (N = 91), Employed part-time (N = 21), Unemployed (N = 31), Other (N = 30)
Occupation	Professional specializing in IT (N = 80, 46%), Student specializing in IT-related fields (N = 53, 30.5%), Others (N = 41, 23.6%)
Years of Programming Experience	Mean = 4.13 years, Standard deviation = 4.365; 25th percentile: 2 years, 50th percentile: 3 years, 75th percentile: 5 years
Years of JavaScript Experience	Mean = 4.13 years, Standard deviation = 4.365; 25th percentile: 1 year, 50th percentile: 1.5 years, 75th percentile: 3 years

4.1.1. Development expertise

We created the latent variable *development expertise* (A_3) in two steps. Firstly, we used Cronbach's alpha (Tavakol and Dennick, 2011) to test whether all the questions used for assessing programmer factors were measuring the same underlying construct. Secondly, we used principal component analysis (PCA) (Cunningham and Ghahramani, 2015) to create the latent variable (B_1) *development expertise* with observed variables B_{11a}, B_{11b}, B_{12a}, B_{12b}, B_{13a}, and B_{13b}.

Cronbach's alpha is a measure of reliability that determines the internal consistency of multiple items designed to measure the same construct (Tavakol and Dennick, 2011). The value of Cronbach's alpha of B_{11a}, B_{11b}, B_{12a}, B_{12b}, B_{13a}, and B_{13b} was 0.833, higher than the cut-off value 0.8 (Tavakol and Dennick, 2011). This result indicated a strong internal consistency among all the scales of observed programmer factors.

We then performed PCA on B_{11a}, B_{11b}, B_{12a}, B_{12b}, B_{13a}, and B_{13b} to compute (A_3) *development expertise*. We tested the linearity assumption and sampling adequacy assumption (Kaiser, 1974) to ensure that the collected data were suitable for PCA. We have tested the Kaiser–Meyer–Olkin (KMO) measure (Kaiser, 1974) and Bartlett's test of sphericity (Tobias and Carlson, 1969) to ensure the sampling adequacy assumption is not violated. The detailed information of KMO measure, Bartlett's test of sphericity, and other detail of PCA analysis is shown in A. We then used the scree plot criterion (Sreejesh et al., 2014) to choose latent factors that represent the observed variables. The scree plot criterion (Sreejesh et al., 2014) suggested that only the first factor should be chosen. Keeping the first component explained 54.729% of the variance, which was acceptable (Sreejesh et al., 2014). The component matrix is shown in Table A.11, indicating that the chosen factor could cover all observed variables. The interpretation of the data was consistent with the concepts of development expertise, since it involves positive coefficients for overall programming experience, programming experience in particular programming languages, and task-specified knowledge familiarity.

4.1.2. Latent code similarity

To simplify the data analysis and see the big picture of code similarity, we used PCA to generate the independent variable *latent code similarity* inferred from the four code similarity metrics listed in Table 3. The procedure of computing latent code similarity was the same as computing development expertise. One latent factor was selected to represent the *TP*, *TA*, *LP*, and *LA*. The retained factor explained 82.550% of the total variances. As shown in Table B.15, the retained latent variable could capture all the features of the four different code similarity metrics. The detailed PCA analysis of code similarity are shown in B.

To validate the suitability of code similarity as a proxy for AI code adoption, we randomly selected 20 pairs of participants' code—specifically, their original submissions before encountering the AI solution and their revised submissions after reviewing the AI-generated code. For each pair, we systematically evaluated two metrics: the similarity between their revised code and the AI solution, and the actual extent of AI code adoption.

Adoption levels were categorized as follows:

Full adoption: The participant entirely discarded their original code and adopted the AI-generated solution in its entirety, with no remnants of their initial implementation retained.

Partial adoption: The participant preserved portions of their original code while incorporating specific components of the AI-generated solution without full replication. The specific components adopted by participants are clarified in the *Details* column in Table 5. Examples include: additions of defensive programming elements (to enhance code robustness, such as incorporating `if (!head) return head` from the AI-generated solution to prevent null pointer errors), and adjustments to control structures (e.g., switching from recursive to iterative implementations to align with the AI solution).

No evidence of adoption: The revised code contains no recognizable elements derived from the AI solution.

As shown in Table 5, our analysis revealed a clear hierarchical pattern: revised code classified as full adoption showed significantly higher similarity to the AI solution than that with partial adoption, while partial adoption cases consistently exhibited greater similarity than those with no adoption. This ordered relationship confirms that code similarity effectively reflects the degree of AI code adoption, thereby validating its use as a reliable proxy measure.

4.2. Hypotheses testing

We employed linear regression to examine the influence of (A_1) *presence of comments* and (B_1) *development expertise* on (Y) *latent code similarity*, using a data analysis approach consistent with previous studies (Lawrie et al., 2006; Kolodziej, 2019). We hypothesize that the level of development expertise exerts a moderation effect (Fairchild and MacKinnon, 2009) on the influence of comments on programmers' adoption of AI-generated code because more experienced programmers may rely less on comments to understand and integrate the code, while less experienced programmers may find comments crucial for comprehension and adoption. To explore the potential moderation effect, we added an interaction term into the linear regression models. The interaction term was calculated as the product of (A_1) *presence of comments* and (B_1) *development expertise*, denoted as $A_1 \times B_1$.

We firstly included (A_1) *presence of comments*, (B_1) *development expertise* and the interaction term $A_1 \times B_1$ into the linear regression. The overall model significantly predicted (Y) *latent code similarity*, $F(3, 173) = 2.724, p = 0.046$. The coefficient table for the factors A_1, B_1, and $A_1 \times B_1$ is shown in Table 6. The results demonstrated that (A_1) the presence of comments could significantly increase programmers' adoption of AI-generated code ($\beta = 0.336, p = 0.027$), supporting hypothesis H_1.[2] We anecdotally observed that comments were copied in the majority of cases and that participants also tended to copy function signatures. A systematic analysis of these behaviors is part of our future work.

[2] To address task-specific variability and uneven dropouts across the five programming tasks, we re-analyzed the data using a mixed-effects model, which included a random intercept for task as the random effect and retained the presence of comments as the fixed effect. The re-analysis confirmed our core findings: the presence of comments remained a significant predictor of AI code adoption ($p = 0.019$), supporting H_1

Table 5
Adoption details of programming tasks.

LeetCode number[a]	Participant's prolific ID[b]	Similarity[c]	Adoption level	Details
83	6511	0.67	partial adoption	defensive programming
83	60cd	0.81	full adoption	
83	5f20	0.95	full adoption	
83	5f0d	−0.01	partial adoption	defensive programming, control structure
98	657d	0.95	full adoption	
98	6525	−0.11	partial adoption	defensive programming
98	650b	0.95	full adoption	
98	60ae	−1.12	no evidence of adoption	
1823	6611	0.95	full adoption	
1823	65e1	−1.21	no evidence of adoption	
1823	64eb	0.95	full adoption	
1823	6129	−0.03	partial adoption	defensive programming
32	654c	0.95	full adoption	
32	627b	0.95	full adoption	
32	5df6	−0.15	partial adoption	defensive programming, control structure
32	5ba0	0.95	full adoption	
383	6558	−1.74	no evidence of adoption	
383	6122	0.95	full adoption	
383	659e	0.95	full adoption	
383	6609	−1.78	no evidence of adoption	

[a] LeetCode Number is a unique identifier assigned to each problem on the LeetCode platform for easy reference and search.
[b] The *Participant's Prolific ID* column displays the first 4 digits of each participant's Prolific ID. This ID serves as the unique identifier for participants on the Prolific platform.
[c] Negative values in the *Similarity* column exist because data was normalized.

Table 6
Coefficients and significance of key factors.

Variable	Coefficient	Standardized coefficient	Significance
(A_1) presence of comments	0.336	0.168	0.027
(B_1) development expertise	−0.113	−0.113	0.253
($A_1 \times B_1$) interaction term	−0.015	−0.010	0.922

As showcased in Table 6, neither (B_1) *development expertise* nor the interaction term between (B_1) *development expertise* and (A_1) *presence of comments* had a significant influence on code adoption ($p > 0.05$). This result indicated that the level of development expertise did not moderate the influence of (A_1) *presence of comments*.[3] Programmers tended to adopt more AI-generated code when comments were present, regardless of their development expertise. The insignificance of B_1 and $A_1 \times B_1$ suggested that (B_1) *development expertise* did not diminish the positive effect of (A_1) *presence of comments* on programmers' adoption of AI-generated code. Therefore, we reject hypothesis H_2.

To further confirm and validate the results in Table 6, we added all control variables, including (B_2) *gender*, (B_3) *age*, (B_{4f}) *fear of AI code generators*, and (B_{4a}) *attitudinal acceptance of AI code generators*, into our initial linear regression model. The coefficients and significance of all variables included in the second model are presented in Table 7. Table 7 indicated that (A_1) *presence of comments* was a significant predictor of (Y) *latent code similarity*, while (B_1) *development expertise* and the interaction term $A_1 \times B_1$ remained insignificant. The second linear regression model confirmed the results of the initial regression model: even when accounting for all control variables, the presence of comments increases programmers' adoption of AI-generated code regardless of programmers' development expertise. Table 7 also showed that all control variables were insignificant, including participants' attitudes towards AI code generators. The discrepancy between self-reported attitudes and actual behavior demonstrates the limitation of overly relying on self-reported intention of adopting AI coding tools, further highlighting the importance of adopting objective measurements of programmers' code adoption behavior.

4.3. Post-hoc analysis

While the latent code similarity variable captures the overall effects of comments and development expertise, we conducted post-hoc analyses to examine their more nuanced effects on individual code similarity metrics—specifically, line-level and token-sequence-level similarities. We re-ran the linear regression models with *TA*, *TP*, *LA*, and *LP* as dependent variables separately. The results of these regression models are presented in C.

These analyses show that for line-level similarities (*LA* and *LP*), the presence of comments (A_1) is not a significant predictor, whereas development expertise (B_1) is. In contrast, for token-sequence-level similarities (*TA* and *TP*), comment presence (A_1) is significant, while development expertise (B_1) is not. This suggests that the global effects observed in our primary model were primarily driven by token-sequence-level similarities.

As introduced in Section 3.4.3, line-level code similarities primarily reflect direct copy-paste behavior, whereas token-sequence-level similarities capture the adoption of structural patterns. A possible explanation for the differing predictors is that less experienced programmers may be more inclined to directly copy and paste lines from the AI-generated solution due to limited confidence or an inability to devise their own approach. In contrast, more experienced programmers may be able to construct a rough solution independently and thus adopt the structural logic of the AI-generated code rather than its exact lines. This behavioral difference may explain why development expertise predicts line-level similarity but not token-level similarity.

[3] To further validate robustness, we re-estimated the model using years of programming experience (highest-loading indicator of development expertise) instead of the composite variable B_1. Results confirmed stability: presence of comments remained significant ($p = 0.037$); other variables remained insignificant (programming experience: $p = 0.801$; interaction term: $p = 0.561$).

Table 7
Coefficients and significance of key factors and control variables.

Variable	Coefficient	Standardized coefficient	Significance
(A_1) presence of comments	0.354	0.177	0.022
(B_1) development expertise	−0.138	−0.138	0.176
($A_1 \times B_1$) interaction term	−0.021	−0.014	0.889
(B_2) gender	0.117	0.008	0.490
(B_3) age	0.009	−0.084	0.401
(B_{4f}) fear of AI code generators	0.003	0.054	0.921
(B_{4a}) attitudinal acceptance of AI code generators	−0.047	0.067	0.280

These findings underscore the importance of being cautious in the operationalization of "code similarity", as different aspects of similarity may be predicted by different factors. Nevertheless, given our study's focus on the broader effects of comments and expertise, we continue to interpret and discuss the latent code similarity variable in the main manuscript, as it captures multiple dimensions of code similarity.

5. Discussion and implications

Our research demonstrates that comments — an essential element in facilitating code comprehension — remain significant in shaping programmers' adoption of AI-generated code within the context of lightweight JavaScript function coding. The presence of comments significantly encourages programmers to adopt AI-generated code, regardless of their development expertise. Both novice and expert programmers tended to adopt more AI-generated code when it included comments.

For novice programmers, the influence of comments on the adoption of AI-generated code can be understood through the Unified Theory of Acceptance and Use of Technology (UTAUT) (Venkatesh et al., 2003). According to UTAUT, technologies that are perceived as easier to use are more likely to be adopted (Venkatesh et al., 2003). Commented code is typically easier to comprehend, and this enhanced comprehensibility can strengthen users' intention to adopt AI-generated solutions.

Previous studies have demonstrated the profound influence of comments on code comprehension (Börstler and Paech, 2016; Kolodziej, 2019). Kolodziej discovered that code snippets with comments significantly decreased the time needed for code comprehension and improved the correctness rates in code comprehension tests (Kolodziej, 2019). Börstler and Paech evaluated the influence of comment quality on code comprehension, and found that even comments of poor quality can lead to a higher perceived code comprehension compared to code snippets without comments (Börstler and Paech, 2016).

The comprehensibility of AI-generated code plays an important role in participants' adoption decisions. Prior studies have shown that many users perceive code understandability as a limitation of AI coding tools (Barke et al., 2023; Vaithilingam et al., 2022; Liang et al., 2024). GitHub Copilot user studies discovered that users found it difficult to understand the code generated by Copilot (Barke et al., 2023; Vaithilingam et al., 2022), which led some to refrain from using it for programming tasks (Barke et al., 2023). In a large-scale survey, Liang et al. found that 16% of participants rated "I don't understand the code written by code generation tools" as "Very important" or "Important" when asked about reasons for not using AI coding tools (Liang et al., 2024). These findings align with UTAUT's emphasis on effort expectancy—users are less likely to adopt a technology they perceive as difficult to use (Venkatesh et al., 2003). When AI-generated code lacks comprehensibility, it increases cognitive load and reduces perceived ease of use, thereby discouraging adoption.

Our study revealed that the presence of comments can also increase the adoption of AI-generated code for expert programmers. Previous studies have discovered that expert programmers were more capable of understanding source code (Lawrie et al., 2006; Wagner and Wyrich, 2022). It is reasonable to extrapolate that expert programmers were more capable of understanding the AI-generated code without comments. However, they were still more likely to adopt AI-generated code that was commented. This indicates that the role of comments extends beyond mere comprehensibility. For expert programmers, the presence of comments may serve as a signal of higher code quality—an aspect they prioritize more strongly due to their elevated expectations and professional standards. Code comments could be leveraged to improve the readability, maintainability, and reliability of software (Tan, 2015). Expert programmers are expected to produce high-quality code that exhibits several desirable characteristics, including enhanced readability and good maintainability (Baltes and Diehl, 2018). Commented code was more capable of meeting the requirements of expert programmers. In this case, their adoption may be more driven by performance expectancy in UTAUT (Venkatesh et al., 2003), specifically the belief that using well-documented AI-generated code will enhance their capability to execute high-quality, maintainable development work.

To encourage more programmers to utilize AI coding tools and enhance the adoption rates of AI-generated code, the developer of AI coding tools should consider ensuring the presence of comments in generated code snippets and increasing the quality of accompanying documentation.

6. Limitations and threats to validity

Construct Validity. The measurement of development expertise might not accurately reflect the true level of participants' development expertise. Development expertise was quantified as a linear combination of (A_{31}) *programming experience*, (A_{32}) *programming language proficiency*, and (A_{33}) *domain knowledge familiarity*. Although we incorporated ostensibly objective measures, such as the number of years participants had focused primarily on programming in their work or studies, these metrics still relied on self-reporting, which can be inherently subjective and potentially unreliable. Baltes and Diehl investigated the validity of self-assessed expertise ratings and years of programming experience among various groups of programmers (Baltes and Diehl, 2018). They concluded that neither the self-evaluated experience ratings nor the years of experience could reliably measure programmers' actual expertise (Baltes and Diehl, 2018).

Measuring AI-generated code adoption through similarity metrics can be objective but may miss some aspects of adoption. While our metrics capture the adoption of code structures and exact lines of code, they overlook the adoption of stylistic aspects, such as identifier naming styles and function declaration styles. Specifically, *TP* and *TA* ignore identifier name changes, and *LP* and *LA* assess only exact line matches. Thus, if a participant adopts naming styles of the AI-generated code, like the camel case naming style, our metrics will not capture this. Additionally, our metrics also overlook cases where programmers may adopt the function declaration style used in AI-generated code, such as favoring arrow functions over traditional function expressions when working with JavaScript.

Another potential constraint on using code similarity as a measure of code adoption is the risk of chance overlap—defined as coincidental similarities between the AI-generated solution and a participant's code

that emerge not from deliberate adoption of the AI output, but because the participant independently developed a solution resembling the AI's. While our analysis of 20 randomly selected code pairs (comparing solutions before and after exposure to the AI) suggested that chance overlap generally does not undermine the metric's validity, residual possibilities remain: in rare cases, such unintended coincidences could potentially compromise the metric's accuracy in reflecting actual adoption.

External Validity. This study faces several threats to external validity. Firstly, the programming language was limited to JavaScript. We chose JavaScript for practical reasons, as using multiple languages would have required significantly more participants and resources. Previous studies have shown that the quality of AI-generated code varies across different languages (Nguyen and Nadi, 2022b). Limiting the study to only one programming language may not accurately capture the overall characteristics of AI-generated code in other languages. Future research should explore how AI-generated comments influence adoption across multiple programming languages, considering that different languages have varying conventions for code readability, documentation practices, and idiomatic structures.

Secondly, the scope of the programming tasks was narrow and restricted to data structure and algorithm problems from LeetCode. We selected LeetCode for feasibility, as it is widely used in related studies (Czarnitzki et al., 2023; Nascimento et al., 2023; Nguyen and Nadi, 2022b,a; Harper, 2023) and provides a familiar, controlled platform for experimentation. Although these problems may not fully represent real-world tasks, they allowed us to cover a range of difficulty levels and topics. This restriction may not accurately represent real-world software engineering contexts, such as web development, system design, and database integration. Additionally, the difficulty of the tasks could influence programmers' adoption of AI-generated code. While we included problems of varying difficulty, we did not analyze whether programmers were more likely to adopt AI-generated code for easier or harder problems. It is possible that programmers struggling with complex problems may rely more heavily on AI-generated solutions, while those solving simpler problems may feel more confident in their own implementations. Future research should systematically explore how task complexity affects AI-generated code adoption.

Thirdly, the complexity of the programming tasks was constrained, with a limited number of possible solutions and short solution lengths, which may not capture the open-ended nature of real-life software development tasks. In professional settings, developers often work on projects with multiple valid approaches, varying levels of abstraction, and domain-specific constraints. Further research could explore AI-generated code adoption in larger-scale software projects where contextual factors such as maintainability, codebase integration, and long-term readability play a role.

Fourthly, the study used error-free AI-generated code, which did not reflect real-world scenarios where AI-generated code can contain errors and vulnerabilities (Pearce et al., 2022; Chen et al., 2021). In real-world applications, developers frequently need to evaluate AI-generated code for correctness and security before adopting it. Additionally, AI-generated code might contain subtle inefficiencies or biases in implementation that could impact long-term maintainability. Future studies could investigate how the presence of AI-generated errors influences adoption behavior, particularly whether programmers with higher expertise are more selective in integrating AI-generated solutions.

Lastly, participants from Prolific are not necessarily representative of programmers, although we tried to mitigate this issue through pre-screening mechanisms. While our participants included those with full-time employment in IT and computer science students, the recruitment method may have introduced biases compared to recruiting professional software engineers working in the industry. Future research could replicate this study with industry practitioners to assess whether similar adoption patterns emerge in workplace settings.

Future studies could enhance the generalizability of the results by expanding the scope and complexity of programming tasks, evaluating the use of erroneous AI-generated code, and recruiting a more diverse group of programmers through professional networks.

Internal Validity. To maintain procedural control, we implemented methods to encourage participants to adhere to the designed procedures. One potential threat is related to the difficulty in preventing participants from copying solutions generated by external AI. To address this, participants were initially asked to solve each problem independently to motivate active engagement and filter out low-effort responses. This self-exclusion approach helped improve data quality by filtering out less meaningful responses.

While acknowledging that external factors could have impacted the code produced, our primary focus remained on adoption behavior, which we believed to be relatively unaffected by these external influences. Even if participants used external AI tools to verify their work, they were still less likely to adopt code without comments, which stands as our central finding, underscoring the influence of comments on code adoption. However, external AI assistance could introduce biases, particularly if participants cross-referenced different AI-generated solutions before submitting their final code. Moreover, different AI tools may produce stylistically distinct solutions, and the AI model used in this study may exhibit implicit biases in generating code. Future studies should examine how different AI code generators influence programmers' adoption behaviors and whether developers prefer AI-generated solutions from specific models based on readability or correctness.

Additionally, real-world software development is often constrained by deadlines, code review processes, and collaboration with other developers, all of which could influence code adoption. Our controlled environment did not fully replicate these constraints, and future research could examine how time pressure and team collaboration affect programmers' willingness to adopt AI-generated code. Furthermore, it remains unclear whether repeated exposure to AI-generated solutions could lead to over-reliance on AI suggestions or whether programmers develop strategies to critically assess AI-generated code over time. Future studies could investigate the long-term effects of AI-assisted coding on software development practices.

7. Conclusion and future work

Our survey-based study explored how the presence of comments and development expertise influence programmers' adoption of AI-generated JavaScript code. Participants reported their expertise, attempted to solve a programming problem independently, and then received AI-generated solutions with or without comments. Adoption was objectively measured through code similarity between participants' submissions and the AI-generated solutions. After data collection, we applied principal component analysis and linear regression to analyze the effects.

Our study has made several important contributions. Firstly, we examined unexplored factors that could potentially affect programmers' adoption of AI-generated code. We found that programmers adopted more AI-generated JavaScript code when comments were present, regardless of development expertise. For industry practitioners, this finding has the potential to motivate developers of AI coding systems to place greater emphasis on annotating AI-generated code. For instance, future training in AI programming systems could incorporate more code with high-quality comments into the training set. This can help create more user-friendly AI coding tools, further boosting the productivity of AI coding assistants. For academia, this finding allows researchers to have a better understanding of the AI-human collaboration paradigm. Future studies could further expand this finding by evaluating what types of comments can help enhance programmers' adoption of AI-generated codes.

Secondly, we proposed novel behavioral measurements for programmers' adoption of AI-generated code, which are valuable complements and alternatives to traditional self-reported data. Rather than directly inquiring of participants about the degree to which they adopted AI-generated code, we employed a behavioral approach. We measured the degree of code adoption by calculating the similarity between the participants' submitted code and the AI-generated code. We evaluated code similarity from two distinct dimensions: the entity perspective and the code similarity level. This new approach could provide behavioral and comprehensive insights about programmers' adoption of AI coding tools. We have also provided the replication package[4] to help future researchers utilize our measurements of code adoption.

Thirdly, we developed a custom survey application that facilitated coding tasks[5]. This survey application incorporated a code editor with advanced features, including syntax highlight and auto-completion, providing a coding experience similar to using VSCode (Del Sole and Sole, 2019). This allowed participants to complete coding tasks in a familiar environment. Future survey-based research can adapt and extend this application to suit their specific requirements, especially if the surveys include coding components.

Future studies could build on our work by addressing its limitations and further validating our findings. One potential direction is adapting our survey application to support additional programming languages. Our app currently integrates the Monaco Editor — the same editor used in VSCode — which provides advanced features like syntax highlighting natively for JavaScript and TypeScript. To offer similar support for other languages, such as Python and Swift, future researchers would need to integrate the Language Server Protocol (LSP, a standardized interface that enables code editors to communicate with language-specific servers, providing consistent advanced features like auto-completion and syntax checking[6]) with the Monaco Editor.

Additionally, future studies could expand our survey application to better reflect real-life programming scenarios. In our current study, participants are provided with pre-generated AI solutions, based on the assumption that they would typically query AI only once for a LeetCode problem. However, in real-world settings, developers often interact with AI tools iteratively, generating multiple solutions using different prompts. To more accurately simulate these interactions, future researchers should enhance the survey app to support on-demand generation of AI solutions at runtime, with the option to include or exclude comments depending on the experimental design.

Future researchers could extend our survey application to collect and analyze richer behavioral data, such as the number of copy and paste operations, to gain deeper insights into how programmers adopt AI-generated code. Specifically, the application could be enhanced to log user interactions, including copy actions, paste actions, and selection changes, to better capture participants' behavior during code adoption.

In addition to further validating our study results, future research could deepen our understanding of how comments influence programmers' adoption of AI-generated code. While there is ample prior research on the impact of comments on code comprehension (Martin, 2009; Takang et al., 1996; Nurvitadhi et al., 2003; Börstler and Paech, 2016; Tan, 2015), future studies could build on these designs to investigate how varying types, lengths, and qualities of comments affect the extent to which programmers adopt AI-generated code.

[4] The data preprocessing scripts, raw data, and the data analysis results can be found at: https://github.com/ArthurLCW/CodeAdoptionResearch/tree/main/analysis

[5] To access the survey app, please visit: https://survey.changwen-software-engineering.xyz; Source code of the survey app can be found at: https://github.com/ArthurLCW/CodeAdoptionResearch/tree/main/survey-web

[6] More information regarding LSP: https://microsoft.github.io/language-server-protocol/

Table A.8
Correlation matrix of observed developer factors.

	B_{11sub}	B_{11obj}	B_{12sub}	B_{12obj}	B_{13sub}	B_{13obj}
B_{11sub}	1.000	0.571	0.445	0.470	0.547	0.527
B_{11obj}	0.571	1.000	0.288	0.748	0.372	0.337
B_{12sub}	0.445	0.288	1.000	0.503	0.347	0.323
B_{12obj}	0.470	0.748	0.503	1.000	0.318	0.290
B_{13sub}	0.547	0.372	0.347	0.318	1.000	0.724
B_{13obj}	0.527	0.337	0.323	0.290	0.724	1.000

Table A.9
KMO measures of observed developer factors.

Observed variable	KMO measure
B_{11sub}	0.824
B_{11obj}	0.632
B_{12sub}	0.669
B_{12obj}	0.636
B_{13sub}	0.742
B_{13obj}	0.734

CRediT authorship contribution statement

Changwen Li: Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Investigation, Formal analysis, Data curation, Conceptualization. **Christoph Treude:** Writing – review & editing, Supervision, Resources, Methodology, Conceptualization. **Ofir Turel:** Supervision, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. PCA results for development expertise

We examined the linearity assumption (Sreejesh et al., 2014) through the correlation matrix in Table A.8. As presented in Table A.8, all observed variables have at least one correlation with another observed variable greater than 0.3. The correlation coefficient is widely used as an indicator of linearity, where the correlation coefficient larger than 0.3 indicates linearity (Asuero and González, 2006). The matrix table indicated that every variable had a linear relationship with some other variables.

We tested the sampling adequacy assumption using the Kaiser–Meyer–Olkin (KMO) measure and Bartlett's test of sphericity. The overall KMO value was 0.703, exceeding the 0.5 threshold (Kaiser, 1974), with all individual KMO values also above 0.5 (see Table A.9), suggesting sufficient shared variance for factor analysis. Bartlett's test of sphericity was significant ($p < 0.001$) (Tobias and Carlson, 1969), indicating that the correlation matrix was not an identity matrix. Together, these results confirmed the data's suitability for factor analysis.

PCA revealed two components with eigenvalues greater than 1, explaining 54.729% and 18.169% of the total variance, respectively. Based on the scree plot criterion (Sreejesh et al., 2014; Cattell, 1966), we retained only the first component, which appears before the inflection point in the scree plot (Fig. A.9). This decision was supported by the high explained variance (54.729%) and the interpretability of the component. We applied Varimax rotation with Kaiser normalization (Kaiser, 1959; Dien et al., 2005), and the rotation matrix (Table A.10) showed that the first component adequately represented all observed variables. We then reran PCA to extract a single component and recalculated the loadings (Table A.11). The resulting component aligned with the concept of Development Expertise (B_1), as it positively

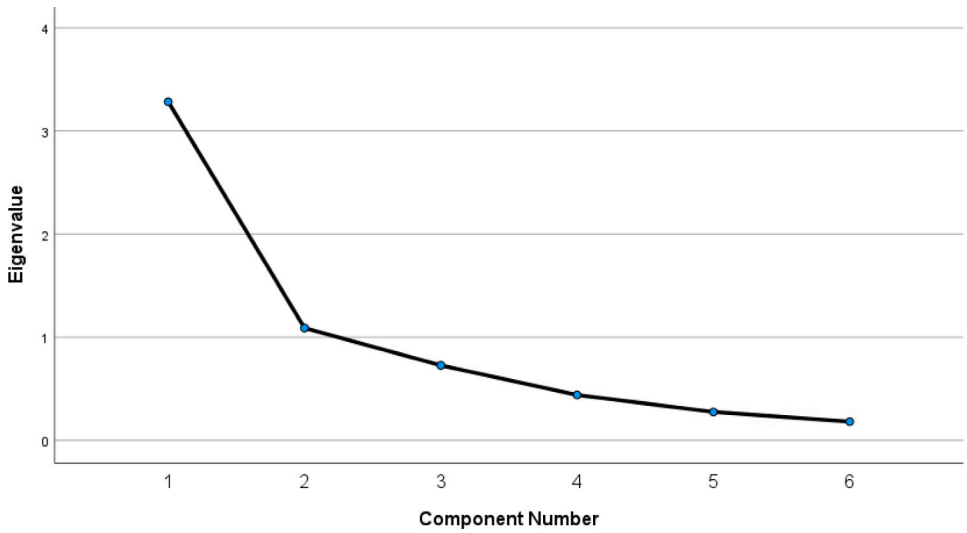

Fig. A.9. Scree plot of principal components for observed programmer factors.

Table A.10
Rotation matrix with component loadings.

Observed variable	Component 1	Component 2
B_{11sub}	0.572	0.315
B_{11obj}	0.922	0.103
B_{12sub}	0.158	0.898
B_{12obj}	0.853	0.189
B_{13sub}	0.205	0.886
B_{13obj}	0.564	0.589

Table A.11
Component matrix for development expertise.

Observed variable	Coefficient
B_{11sub}	0.751
B_{11obj}	0.813
B_{12sub}	0.748
B_{12obj}	0.758
B_{13sub}	0.722
B_{13obj}	0.635

Table A.12
Communalities for observed variables in development expertise.

Observed variable	Communality
B_{11sub}	0.665
B_{11obj}	0.763
B_{12sub}	0.427
B_{12obj}	0.861
B_{13sub}	0.827
B_{13obj}	0.832

loaded on general programming experience, language proficiency, and domain-specific knowledge.

Table A.12 reports the communalities for each observed variable included in the PCA. These values represent the proportion of variance in each variable accounted for by the extracted principal component (Mooi et al., 2018). Most variables show high communalities (e.g., $B_{12obj} = 0.861$; $B_{13sub} = 0.827$), indicating they are well explained by the component. This supports the validity of summarizing these variables into a single latent factor representing development expertise (B_1).

Table B.13
Correlation matrix of code similarity metrics.

	TP	TA	LP	LA
TP	1.000	0.962	0.669	0.628
TA	0.962	1.000	0.687	0.688
LP	0.669	0.687	1.000	0.969
LA	0.628	0.688	0.969	1.000

Table B.14
KMO measures of code similarity metrics.

Code similarity metric	KMO measure
TP	0.515
TA	0.538
LP	0.537
LA	0.516

Table B.15
Component matrix for latent code similarity.

Code similarity metric	Coefficient
TP	0.915
TA	0.905
LP	0.896
LA	0.918

Table B.16
Communalities for code similarity metrics.

Code similarity metric	Extraction
TP	0.838
TA	0.818
LP	0.803
LA	0.843

Appendix B. PCA results for code similarity

Similar to the process for generating *development expertise* (B_1), we first tested the linearity and sampling adequacy assumptions. Linearity was assessed using the correlation matrix in Table B.13. All code similarity metrics showed at least one correlation above 0.6, with several exceeding 0.9 (e.g., between *TP* and *TA*; *LP* and *LA*), indicating strong inter-correlations (Asuero and González, 2006). These high correlations satisfy the linearity assumption and suggest the metrics are closely related. The strong inter-correlations may be attributed to the short

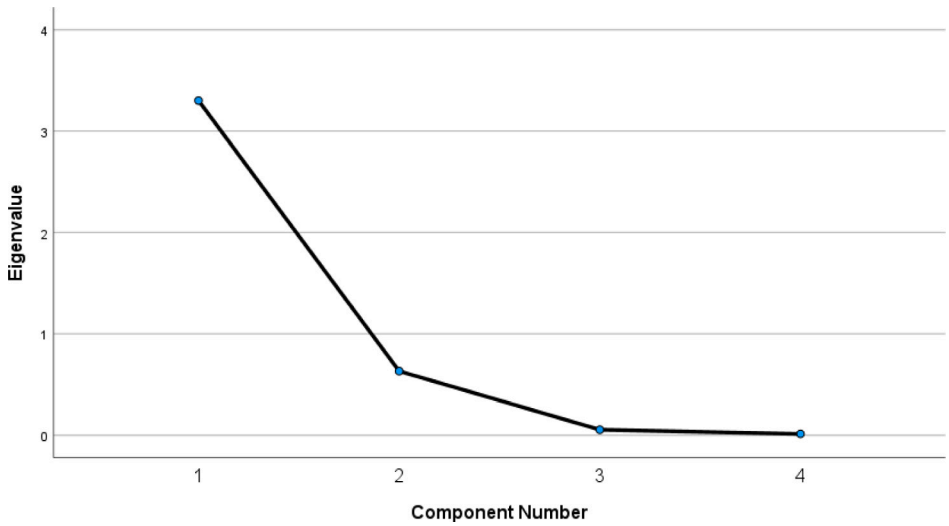

Fig. B.10. Scree plot of principal components for code similarity metrics.

Table C.17
Regression results with *TA* as dependent variable.

Variable	Coefficient	Standardized coefficient	*p*-value
(A_1) Presence of comments	0.157	0.208	0.006
(B_1) Development expertise	−0.012	0.012	0.835
($A_1 \times B_1$) Interaction term	−0.021	0.005	0.900

Table C.18
Regression results with *TP* as dependent variable.

Variable	Coefficient	Standardized coefficient	*p*-value
(A_1) Presence of comments	0.118	0.156	0.041
(B_1) Development expertise	−0.002	−0.004	0.964
($A_1 \times B_1$) Interaction term	−0.001	−0.003	0.980

Table C.19
Regression results with *LA* as dependent variable.

Variable	Coefficient	Standardized coefficient	*p*-value
(A_1) Presence of comments	0.093	0.138	0.064
(B_1) Development expertise	−0.070	−0.207	0.034
($A_1 \times B_1$) Interaction term	−0.011	−0.020	0.833

Table C.20
Regression results with *LP* as dependent variable.

Variable	Coefficient	Standardized coefficient	*p*-value
(A_1) Presence of comments	0.070	0.109	0.147
(B_1) Development expertise	−0.068	0.004	0.032
($A_1 \times B_1$) Interaction term	−0.211	0.009	0.930

length of the code solutions (typically under 25 lines), which limits structural variability and amplifies similarity across metrics.

After testing linearity, we assessed sampling adequacy. The overall Kaiser–Meyer–Olkin (KMO) measure was 0.527, with all individual KMO values above 0.5 (Table B.14), indicating sufficient shared variance for dimensionality reduction (Kaiser, 1974). Bartlett's test of sphericity was significant ($p < 0.001$), suggesting the correlation matrix was not an identity matrix (Tobias and Carlson, 1969). Together, these results confirmed that the sampling adequacy assumption was met.

PCA revealed one principal component with an eigenvalue above 1, explaining 82.550% of the total variance. The scree plot (Fig. B.10) confirmed that only the first component should be retained, as the second point marked the inflection point. This component captured the core variance across all four code similarity metrics and was interpretable as a unified latent variable. We used it as the dependent variable, with its component loadings shown in Table B.15.

We examined the communalities to assess how much of each code similarity metric's variance is captured by the extracted principal component. As shown in Table B.16, all four code similarity metrics had high extraction values, ranging from 0.803 to 0.843. This indicates that the retained component effectively represents the shared variance across these metrics, supporting the dimensionality reduction.

Appendix C. Post-hoc regression analyses on individual code similarity metrics

We conducted post-hoc regression analyses to examine how development expertise (B_1) and the presence of comments (A_1) influence individual code similarity metrics. Specifically, we analyzed: token-sequence-level similarity from the participant's perspective (*TP*), token-sequence-level similarity from the AI's perspective (*TA*), line-level similarity from the participant's perspective (*LP*), and line-level similarity

from the AI's perspective (*LA*). We employed the same linear regression model described in Section 4.2, replacing the latent code similarity variable with each of the four individual similarity metrics as dependent variables. The regression results are summarized in Tables C.17–C.20.

Data availability

I have provided the link to the data and codes both in the manuscript and within the submission form.

References

Amoozadeh, M., Daniels, D., Nam, D., Kumar, A., Chen, S., Hilton, M., Srinivasa Ragavan, S., Alipour, M.A., 2024. Trust in generative AI among students: An exploratory study. In: Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1. SIGCSE 2024, Association for Computing Machinery, New York, NY, USA, pp. 67–73. http://dx.doi.org/10.1145/3626252.3630842.

Asuero, A.S.A.G., González, A.G., 2006. The correlation coefficient: An overview. Crit. Rev. Anal. Chem. 36 (1), 41–59. http://dx.doi.org/10.1080/10408340500526766.

Bader, J., Seohyun Kim, S., Sifei Luan, F., Chandra, S., Meijer, E., 2021. AI in software engineering at facebook. IEEE Softw. 38 (4), 52–61. http://dx.doi.org/10.1109/MS.2021.3061664.

Baltes, S., Diehl, S., 2018. Towards a theory of software development expertise. In: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ESEC/FSE 2018, Association for Computing Machinery, New York, NY, USA, pp. 187–200. http://dx.doi.org/10.1145/3236024.3236061.

Bandura, A., 1989. Human agency in social cognitive theory. Am. Psychol. 44 (9), 1175.

Barke, S., James, M.B., Polikarpova, N., 2023. Grounded copilot: How programmers interact with code-generating models. Proc. ACM Program. Lang. 7 (OOPSLA1), http://dx.doi.org/10.1145/3586030.

Börstler, J., Paech, B., 2016. The role of method chains and comments in software readability and comprehension—an experiment. IEEE Trans. Softw. Eng. 42 (9), 886–898.

Brase, G.L., 2009. How different types of participant payments alter task performance. Judgm. Decis. Mak. 4 (5), 419–428.

Cattell, R.B., 1966. The scree test for the number of factors. Multivar. Behav. Res. 1 (2), 245–276.

Chan, D., 2010. So why ask me? Are self-report data really that bad? In: Statistical and Methodological Myths and Urban Legends. Routledge, pp. 329–356.

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H.P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F.P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W.H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A.N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., Zaremba, W., 2021. Evaluating large language models trained on code. arXiv:2107.03374.

Cunningham, J.P., Ghahramani, Z., 2015. Linear dimensionality reduction: Survey, insights, and generalizations. J. Mach. Learn. Res. 16 (1), 2859–2900.

Czarnitzki, D., Fernández, G.P., Rammer, C., 2023. Artificial intelligence and firm-level productivity. J. Econ. Behav. Organ. 211, 188–205. http://dx.doi.org/10.1016/j.jebo.2023.05.008, URL https://www.sciencedirect.com/science/article/pii/S0167268123001531.

Del Sole, A., Sole, D., 2019. Visual Studio Code Distilled. Springer.

DeSimone, J.A., DeSimone, A.J., Harms, P.D., Wood, D., 2018. The differential impacts of two forms of insufficient effort responding. Appl. Psychol. 67 (2), 309–338.

Dien, J., Beal, D.J., Berg, P., 2005. Optimizing principal components analysis of event-related potentials: matrix type, factor loading weighting, extraction, and rotations. Clin. Neurophysiol. 116 (8), 1808–1825.

Donaldson, S.I., Grant-Vallone, E.J., 2002. Understanding self-report bias in organizational behavior research. J. Bus. Psychol. 17, 245–260.

Fairchild, A.J., MacKinnon, D.P., 2009. A general model for testing mediation and moderation effects. Prev. Sci. 10, 87–99.

Feng, Y., Vanam, S., Cherukupally, M., Zheng, W., Qiu, M., Chen, H., 2023. Investigating code generation performance of chat-GPT with crowdsourcing social data. In: Proceedings of the 47th IEEE Computer Software and Applications Conference. pp. 1–10.

García-Avilés, J.-A., 2020. Diffusion of innovation. Int. Encycl. Media Psychol. 1–8.

Ge, H., Wu, Y., 2023. An empirical study of adoption of ChatGPT for bug fixing among professional developers. Innov. Technol. Adv. 1 (1), 21–29.

Harper, J., 2023. A Software Engineer's Guide to Seniority: A Guide to Technical Leadership, first ed. Apress, Berkeley, CA, p. IX, 63. http://dx.doi.org/10.1007/978-1-4842-8783-5, Published: 02 December 2022.

Kaiser, H.F., 1959. Computer program for varimax rotation in factor analysis. Educ. Psychol. Meas. 19 (3), 413–420.

Kaiser, H.F., 1974. An index of factorial simplicity. Psychometrika 39 (1), 31–36.

Kolodziej, S., 2019. Empirical assessment of software documentation strategies: A randomized controlled trial. In: Proceedings of the 50th ACM Technical Symposium on Computer Science Education. pp. 1296–1295.

Lancaster, T., Culwin, F., 2004. A comparison of source code plagiarism detection engines. Comput. Sci. Educ. 14 (2), 101–112.

Lawrie, D., Morrell, C., Feild, H., Binkley, D., 2006. What's in a name? A study of identifiers. In: 14th IEEE International Conference on Program Comprehension. ICPC'06, pp. 3–12. http://dx.doi.org/10.1109/ICPC.2006.51.

Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Lago, A.D., Hubert, T., Choy, P., de Masson d'Autume, C., Babuschkin, I., Chen, X., Huang, P.-S., Welbl, J., Gowal, S., Cherepanov, A., Molloy, J., Mankowitz, D.J., Robson, E.S., Kohli, P., de Freitas, N., Kavukcuoglu, K., Vinyals, O., 2022. Competition-level code generation with AlphaCode. Science 378 (6624), 1092–1097. http://dx.doi.org/10.1126/science.abq1158, arXiv:https://www.science.org/doi/pdf/10.1126/science.abq1158. URL https://www.science.org/doi/abs/10.1126/science.abq1158.

Liang, J.T., Yang, C., Myers, B.A., 2024. A large-scale survey on the usability of AI programming assistants: Successes and challenges. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24, Association for Computing Machinery, New York, NY, USA, http://dx.doi.org/10.1145/3597503.3608128.

Logg, J.M., Minson, J.A., Moore, D.A., 2019. Algorithm appreciation: People prefer algorithmic to human judgment. Organ. Behav. Hum. Decis. Process. 151, 90–103. http://dx.doi.org/10.1016/j.obhdp.2018.12.005, URL https://www.sciencedirect.com/science/article/pii/S0749597818303388.

Manna, Z., Waldinger, R.J., 1971. Toward automatic program synthesis. Commun. ACM 14 (3), 151–165. http://dx.doi.org/10.1145/362566.362568.

Marangunić, N., Granić, A., 2015. Technology acceptance model: a literature review from 1986 to 2013. Univers. Access inf. Soc. 14, 81–95.

Martin, R.C., 2009. CLean Code: a Handbook of Agile Software Craftsmanship. Pearson Education.

Microsoft, 2024. Monaco - the editor of the web. https://microsoft.github.io/monaco-editor/. Online. (Accessed 26 June 2024).

Mohagheghi, P., Conradi, R., 2007. Quality, productivity and economic benefits of software reuse: a review of industrial studies. Empir. Softw. Eng. 12, 471–516.

Mooi, E., Sarstedt, M., Mooi-Reci, I., 2018. Principal component and factor analysis. In: Market Research: The Process, Data, and Methods using Stata. Springer Singapore, Singapore, pp. 265–311. http://dx.doi.org/10.1007/978-981-10-5218-7_8.

Nascimento, N., Alencar, P., Cowan, D., 2023. Artificial intelligence vs. Software engineers: An empirical study on performance and efficiency using ChatGPT. In: Proceedings of the 33rd Annual International Conference on Computer Science and Software Engineering. CASCON '23, IBM Corp., USA, pp. 24–33.

Nguyen, N., Nadi, S., 2022a. An empirical evaluation of GitHub copilot's code suggestions. In: Proceedings of the 19th International Conference on Mining Software Repositories. MSR '22, Association for Computing Machinery, New York, NY, USA, pp. 1–5. http://dx.doi.org/10.1145/3524842.3528470.

Nguyen, N., Nadi, S., 2022b. An empirical evaluation of GitHub copilot's code suggestions. In: Proceedings of the 19th International Conference on Mining Software Repositories. MSR '22, Association for Computing Machinery, New York, NY, USA, pp. 1–5. http://dx.doi.org/10.1145/3524842.3528470.

Nurvitadhi, E., Leung, W.W., Cook, C., 2003. Do class comments aid java program understanding? FIE 2003, In: 33rd Annual Frontiers in Education, 2003, vol. 1, http://dx.doi.org/10.1109/FIE.2003.1263332, T3C–T3C.

Palan, S., Schitter, C., 2018. Prolific. ac—A subject pool for online experiments. J. Behav. Exp. Financ. 17, 22–27.

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., Karri, R., 2022. Asleep at the keyboard? assessing the security of github copilot's code contributions. In: 2022 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 754–768.

Prechelt, L., Malpohl, G., Philippsen, M., et al., 2002. Finding plagiarisms among a set of programs with JPlag. J. Univers. Comput. Sci. 8 (11), 1016.

Russo, D., 2024. Navigating the complexity of generative ai adoption in software engineering. ACM Trans. Softw. Eng. Methodol. 33 (5), 135. http://dx.doi.org/10.1145/3652154, https://doi.org/10.1145/3652154.

Russo, D., Hanel, P.H., Altnickel, S., Van Berkel, N., 2021. The daily life of software engineers during the covid-19 pandemic. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice. ICSE-SEIP, IEEE, pp. 364–373.

Russo, D., Stol, K.-J., 2020. Gender differences in personality traits of software engineers. IEEE Trans. Softw. Eng. 48 (3), 819–834.

Schild, C., Lilleholt, L., Zettler, I., 2021. Behavior in cheating paradigms is linked to overall approval rates of crowdworkers. J. Behav. Decis. Mak. 34 (2), 157–166.

Schleimer, S., Wilkerson, D.S., Aiken, A., 2003. Winnowing: local algorithms for document fingerprinting. In: Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data. SIGMOD '03, Association for Computing Machinery, New York, NY, USA, pp. 76–85. http://dx.doi.org/10.1145/872757.872770.

Sindermann, C., Sha, P., Zhou, M., Wernicke, J., Schmitt, H.S., Li, M., Sariyska, R., Stavrou, M., Becker, B., Montag, C., 2021. Assessing the attitude towards artificial intelligence: Introduction of a short measure in german, Chinese, and english language. KI-Künstliche Intell. 35, 109–118.

Sreejesh, S., Mohapatra, S., Anusree, M., Sreejesh, S., Mohapatra, S., Anusree, M., 2014. Exploratory factor and principal component analysis. Bus. Res. Methods: Appl. Orientat. 207–228.

Takang, A.A., Grubb, P.A., Macredie, R.D., et al., 1996. The effects of comments and identifier names on program comprehensibility: an experimental investigation. J. Prog. Lang. 4 (3), 143–167.

Tan, L., 2015. Code comment analysis for improving software quality. In: The Art and Science of Analyzing Software Data. Elsevier, pp. 493–517.

Tavakol, M., Dennick, R., 2011. Making sense of Cronbach's alpha. Int. J. Med. Educ. 2, 53.

Tobias, S., Carlson, J.E., 1969. Brief report: Bartlett's test of sphericity and chance findings in factor analysis. Multivar. Behav. Res. 4 (3), 375–377.

Vaithilingam, P., Zhang, T., Glassman, E.L., 2022. Expectation vs. Experience: Evaluating the usability of code generation tools powered by large language models. In: Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems. CHI EA '22, Association for Computing Machinery, New York, NY, USA, http://dx.doi.org/10.1145/3491101.3519665.

Van de Mortel, T.F., 2008. Faking it: social desirability response bias in self-report research. Aust. J. Adv. Nurs. 25 (4), 40–48.

Venkatesh, V., Morris, M.G., Davis, G.B., Davis, F.D., 2003. User acceptance of information technology: Toward a unified view. MIS Q. 425–478.

Wagner, S., Wyrich, M., 2022. Code comprehension confounders: A study of intelligence and personality. IEEE Trans. Softw. Eng. 48 (12), 4789–4801. http://dx.doi.org/10.1109/TSE.2021.3127131.

Xu, F.F., Vasilescu, B., Neubig, G., 2022. In-IDE code generation from natural language: Promise and challenges. ACM Trans. Softw. Eng. Methodol. 31 (2), http://dx.doi.org/10.1145/3487569.